

## Line Following Robot

Students:

1. Ramez AlMasadeh
2. Abdullah AlBakri
3. Ahmad Alamir
4. Yazan Kanakri

## Main Features

### Finite State Machine (FSM)

Normal tracking → PID line following

Turn FSM

- ACQUIRE: fast pivot to find the new line.
- CENTERING: slow pivot to settle accurately.

Obstacle Avoidance FSM

- AVOID\_RIGHT → pivot away from obstacle.
- AVOID\_FORWARD → short forward bypass.
- AVOID\_LEFT → re-align with the track.

### PID Controller

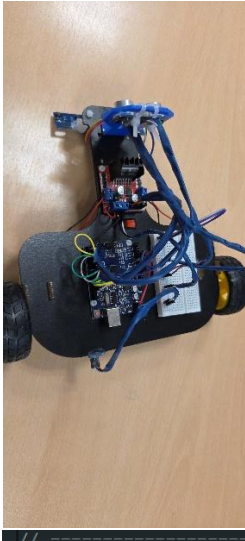
Maintain the robot centered on the line during normal motion ( $K_p$ ,  $k_i$ ,  $k_d$  values are in the pictures).

### Exponential Moving Average (EMA) on each line sensor

- Smooth error signal
- Stable PID behavior
- Reliable derivative term
- Improved curve handling

### Safety & Robustness Features

- Non-blocking timing (no delay()) in main loop)
- FSM overrides to prevent conflicting behaviors
- Speed limits and output constraints



```
int BASE_SPEED = 125;
int SLIGHT_SPEED = 110;

int PIVOT_ACQUIRE_SPEED = 155; //
int PIVOT_CENTER_SPEED = 135; //

int MAX_SPEED = 180;
int MIN_SPEED = 80;

// =====
// PID
// =====
float Kp = 16;
float Ki = 0.1;
float Kd = 12;

float error = 0; // INITIAL VALUE FOR ERROR
float lastError = 0;
float integral = 0;

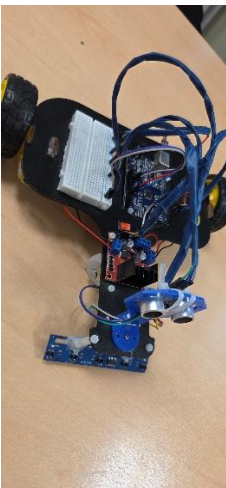
// =====
// TURN STATE MACHINE
// =====
enum TurnState { TURN_NONE, TURN_LEFT, TURN_RIGHT };
enum TurnPhase { ACQUIRE, CENTERING };

TurnState turnState = TURN_NONE;
TurnPhase turnPhase = ACQUIRE;
```

```
// =====
// OBSTACLE AVOIDANCE
// =====
enum AvoidPhase { AVOID_NONE, AVOID_RIGHT, AVOID_FORWARD, AVOID_LEFT };
AvoidPhase avoidPhase = AVOID_NONE;
unsigned long avoidStartMs = 0;

const int OBST_CM = 5; // distance threshold
const unsigned long AVOID_R_MS = 320; // small right pivot
const unsigned long AVOID_F_MS = 450; // short forward
const unsigned long AVOID_L_MS = 490; // small left pivot

const unsigned long OBST_CHECK_MS = 60; // don't spam ultrasonic
unsigned long lastObstCheckMs = 0;
```



```
void forwardPID() {
    error = getError() / 2.0;

    if (abs(error) < 1) error = 0;

    integral += error;
    integral = constrain(integral, -60, 60); // constrained because the robot in dashes has error is large

    float derivative = constrain(error - lastError, -2, 2);
    lastError = error;

    float pid = Kp * error + Ki * integral + Kd * derivative;
    pid = constrain(pid, -35, 35);

    int leftSpeed = (int)(BASE_SPEED - pid);
    int rightSpeed = (int)(BASE_SPEED + pid);

    leftSpeed = constrain(leftSpeed, MIN_SPEED, MAX_SPEED);
    rightSpeed = constrain(rightSpeed, MIN_SPEED, MAX_SPEED);

    analogWrite(ENA, leftSpeed);
    analogWrite(ENB, rightSpeed);
}
```